

Agenda.

→ BUILDER.

↳ Creational Design Pattern

Usecase

→ Class with lot of attributes.

Class Student {

- Name
 - age
 - batch
 - Psp
 - UnivName
 - Company
 - gradYear
 - phoneNo
- ==
}

3

⇒ Student st = new Student();

st.setName(-);

st.setAge(-);

st.setBatch(-);

==
==
==
==

→ Validate the object before its creation.

⇒ No object of Student should be created if any validation fails.

Validations.

1) gradYear $\geq$ 2021.

2) phoneNo should be valid.

Class Student {

- Name
- age
- batch
- Psp
- UnivName
- Company
- gradYear
- phoneNo

Student (name, age, batch, psp, UnivName,
Company,) {

Validations {

- if (grad < 2021) {
 throw RuntimeException(---);
}
- if (phoneNo ---) {

- }

```

this.name = name;
this.age = age;
this.batch = ____;
____
____
____

```

3

3

```

Student st = new Student("Rohit", 24, "Apr 24 Beg.
Morning", 89.45, -, - - - -);

```

→ Difficult to understand.

→ Prone to Errors.

⇒ Combination of Constructors.

```

Student (String name, int age) {
    this.name = name;
    this.age = age;
}

```

3

```

Student (String name, int age, String univName) {
    this.name = name;
    this.age = age;
    this.univName = univName
}

```

3

```
Student (String name) {  
    this.name = name;  
}
```

3

N attrs. $\Rightarrow$ 2^N Constructors.

$\Rightarrow$ Too many constructors.

```
Student (String name, int age) {  
    this.name = name;  
    this.age = age;  
}
```

3

```
Student (String name, int gradYear) {  
    this.name = name;  
    this.gradYear = gradYear;  
}
```

3

$\rightarrow$ Student (String, int)
 $\rightarrow$ Student (String, int) } Same signature

$\Rightarrow$ Some of the constructors won't be possible.

⇒ Lot of code duplication.

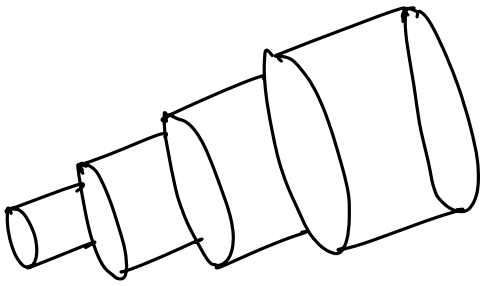
3
Student (String name, int age) {
 this.name = name;
 this.age = age;
}

3
Student (String name, int age, String univName) {
 this.name = name;
 this.age = age;
 this.univName = univName
}

3
Student (String name) {
 this.name = name;
}

3
Student (String name, int age, String univName) {
 this(name, age);
 this.univName = univName
}

⇒ Telescopic Constructor.



⇒ Requirement :

Student (param)



Some data structure that allows us to pass multiple values with different data type.

"name" : "Rohit"

"age" : 24

"batch" : "Apr 24 Beginner"

"psp" : 92.15

} K, V

⇒ HashMap { String, Object }

Student {

====

Student(HashMap<String, Object> map) {
(====) validations.

this.name = (String) map.get("name");

this.age = (Integer) map.get("age");
====

====

3

3

⇒ map.put("nama", "Rohit");

↳ Typo in the key.

map.put("age", "Scaler");

↳ INT ⇒ Runtime Exception.

Map: No restriction on the key
+

No compile time safety on the value.

```
HashMap<String, Object> map = _____;
```

```
map.put("name", "Rohit");
```

```
_____  
_____  
_____  
_____
```

```
Student st = new Student(map);
```

Student (param)



Something like Map that allows to store different keys with their values but there should be compile time safety.

~~x. name~~

~~x. age = "Scaler"~~

Builder

```
Class Helper {  
    - Name ✓ string  
    - age ← int  
    - batch ← string  
    - Psp ← Double  
    - UnivName  
    - Company  
    - gradYear  
    - phoneNo  
}
```

3

```
Helper helper = new Helper()
```

```
helper.setName("Rohit")
```

```
helper.setAge(24);
```

→ Type safety ✓
→ No typo in attrs ✓

```
Student(helper) {
```

Validations

```
{  
    if (helper.getGradYear() < 2021) {  
        ==  
    }  
    3  
    ==  
    ==  
    ==  
}
```

```
this.name = helper.getName();  
    ==  
    ==  
    ==  
}
```

113

BUILDER PATTERN.

Allows us to create an object of a class with following constraints:

- 1) Class with too many parameters.
- 2) Validate the object before creation.